

AgentShield: Mid-Sem Review Presentation Dossier

5-Slide Executive Academic Deck & Evaluation Criteria Assessment
Aligned with VIT FF No. 180 Progress Review 1 Parameters

ACADEMIC & INSTITUTIONAL PROFILE

Institution:	Vishwakarma Institute of Technology (VIT), Pune
Department:	Department of Computer Engineering (Academic Year 2026-27)
Semester / Group:	Semester 5 Group No.: TYG19 Area: AI & Cybersecurity
Project Title:	AgentShield: A Dynamic Risk-Aware Security Framework for the Model Context Protocol
Faculty Guide:	Prof. Sanyukta Deshmukh (Contact: 9021907414, Sanyukta.deshmukh@vit.edu)
Form Reference:	Issue 01 : Rev No. 00 : Dt. 05/02/26 (FF No. 180 Progress Review 1)

GROUP MEMBERS & INDIVIDUAL RESPONSIBILITIES

Sr	Name	Div/Roll	G.R. No.	Email ID	Individual Technical Responsibility
1	Harsh Manjramkar	TY-G/70	12414557	harsh.manjramkar24@vit.edu	Lead Architect, Async Proxy Gateway Core & JSON-RPC Router
2	Vedant Gaidhani	TY-G/74	12415020	vedant.gaidhani242@vit.edu	Dynamic 3-Tier Risk Classifier & Heuristic Injection Detectors
3	Vineet Wagh	TY-G/75	12415021	vineet.wagh241@vit.edu	Response PII Redaction Engine & Rate Limit Loop Quarantine
4	Arjun Joshi	TY-G/76	12415022	arjun.joshi241@vit.edu	Async Batch Audit Logger & Real-time WebSockets Dashboard

EVALUATION PARAMETERS MAPPING TABLE (VIT ASSESSMENT SCHEME)

1. Problem Definition, Related Work & Complexity	[Slide 1 & Slide 2]	Autonomous AI risk, prompt injection, lack of oversight, <5ms latency com
2. Literature Review, Proposed Solution & Feasibility	[Slide 2]	5 foundational papers evaluated, runtime proxy solution, FastAPI ASGI fea
3. Cost, Resources, Environment & Sustainability	[Slide 4]	100% open-source, edge-compatible, prevents wasteful LLM loops (green
4. Group Formation & Individual Responsibilities	[Slide 1 & Cover]	Clear task split across Gateway, Risk Engine, PII Sanitizer, and Audit/UI.
5. Objectives of the Project	[Slide 1]	8 specific measurable goals aligned with FF No. 180 Section 4.
6. System Architecture & Engineering Pipeline	[Slide 3]	10-stage sequential data pipeline (Interception -> Risk -> HITL -> Sanitizer
7. Methodology & Defense In-Depth Strategy	[Slide 3]	Regex automata, argument heuristics, sliding window limiter, recursive JS
8. Knowledge of Domain, Technology & Tools	[Slide 3 & Slide 5]	FastAPI, JSON-RPC 2.0, Async SQLAlchemy, WebSockets, 15 real MCP
9. Progress Done Till Now vs. End-Sem Roadmap	[Slide 5]	Core proxy & security modules operational; public demo portal & container
10. EDI Guide Consultation Points	[Slide 5]	Enterprise RBAC, live public demo portal & package publishing as advised

SLIDE 1 OF 5: Project Overview, Objectives, Team Responsibilities & Problem Complexity

Parameters Covered: Problem Definition, Related Work, Complexity, Group Formation, Project Objectives

Project Context & Domain

- Title: AgentShield: A Dynamic Risk-Aware Security Framework for the Model Context Protocol.
- Academic Profile: Group TYG19 | Computer Engineering | AY 2026-27 | Internal Guide: Prof. Sanyukta Deshmukh.
- Team Roles: Harsh (Gateway/Router), Vedant (Risk/Injection), Vineet (PII/RateLimit), Arjun (Audit/Dashboard).

Problem Definition & Architectural Complexity

- Problem Statement: Direct point-to-point integration of AI agents with enterprise tools (databases, filesystems, shell) via MCP introduces critical runtime vulnerabilities due to hallucinations, prompt injections, and lack of human supervision.
- Complexity: The security gateway must inspect bidirectional, deeply-nested JSON-RPC payloads in real time, evaluate multi-variable risk heuristics, and hold high-risk calls synchronously while maintaining sub-5 millisecond proxy latency.

Core Project Objectives (FF No. 180 Aligned)

1. Intercept & inspect all JSON-RPC 2.0 communication transparently without modifying agent or server code.
2. Implement a Dynamic Risk Engine classifying requests into LOW, MEDIUM, and HIGH risk categories.
3. Detect prompt injections, DAN jailbreaks, SQLi, and sensitive command execution (sudo, chmod, /etc/).
4. Enforce real-time Human-in-the-Loop (HITL) approval gates with 60-second fail-closed timeouts.
5. Recursively sanitize target MCP responses to prevent PII (AWS keys, Aadhaar, PAN) from leaking into LLMs.

SPEAKER NOTES & FACULTY EXAMINER TALKING POINTS:

Good morning respected guide Prof. Sanyukta Deshmukh and committee. Slide 1 establishes our team allocation, formal problem definition, and objectives. MCP grants AI direct tool execution rights without a firewall. AgentShield provides the missing runtime guardrail without adding perceptible latency.

SLIDE 2 OF 5: Literature Review, Related Work & Proposed Solution Feasibility

Parameters Covered: Literature Review, Related Work, Proposed Solution, Technical Approach, Feasibility

Literature Survey & Critical Gaps (5 Foundational Papers)

- MCPShield (2026): Evaluates historical server trust; Limitation: Static pre-handshake checks only, lacks runtime inspection.
- MCPSec (2026): Protocol-level encryption and message integrity; Limitation: Cannot inspect or govern semantic agent actions.
- ShieldMCP (2026): Basic proxy interceptor; Limitation: Lacks dynamic contextual risk scoring, response PII sanitization, and audit.
- Enterprise Security Framework (2025): Governance guidelines; Limitation: Conceptual policies without runtime implementation.
- Security Threats Analysis (2026): Threat taxonomy; Limitation: Focuses on attack discovery rather than runtime mitigation.

Proposed Solution: Transparent Runtime Security Gateway

- Architecture: AgentShield acts as a smart reverse proxy between AI clients (OpenAI/Claude) and target MCP servers.
- Dynamic Risk Governance: Replaces static rules with multi-factor risk assessment evaluating tools, arguments, and agent history.
- Zero-Trust Human Oversight: Pauses dangerous commands for administrator sign-off before execution reaches enterprise systems.

Technical Approach & Engineering Feasibility

- Feasibility: Standard JSON-RPC 2.0 compliance means any client configuring proxy URL (`http://proxy:8000/mcp/v1/proxy`) works instantly.
- Low-Latency Execution: Built on asynchronous ASGI (FastAPI + Starlette) with non-blocking I/O, ensuring negligible proxy overhead.

SPEAKER NOTES & FACULTY EXAMINER TALKING POINTS:

Slide 2 presents our literature study. Existing 2025-2026 research focuses purely on transport encryption or static server trust. AgentShield bridges this gap with an active runtime proxy that evaluates semantics, pauses for human approval, and sanitizes responses.

SLIDE 3 OF 5: System Architecture, Methodology & Technology Stack

Parameters Covered: System Architecture, Methodology, Knowledge of Domain, Technology and Tools

System Architecture & 10-Stage Request Lifecycle

- 1. Inbound Interceptor -> Captures JSON-RPC 2.0 requests at /mcp/v1/proxy.
- 2. Dynamic Risk Classifier -> Analyzes method, tool name, and payload; assigns LOW, MEDIUM, or HIGH score.
- 3. Heuristic Detectors -> Scans arguments for injection tags (<system_override>), SQLi, and shell commands (sudo, chmod).
- 4. Decision Gate -> LOW: Forward immediately | INJECTION: Block (-32002) | SENSITIVE TOOL: Hold for HITL (-32001).
- 5. Rate Limiter / Quarantine -> Sliding-window tracker isolates looping agents (>10 calls in 5s) for 60s.
- 6. Outbound Forwarder -> Asynchronous HTTPX client forwards approved requests to target MCP servers on port 8001.
- 7. Response PII Sanitizer -> Deep recursive object walker redacts AWS keys, Aadhaar numbers, and credentials.
- 8. Async Audit Logger -> Non-blocking asyncio.Queue batches transaction records directly to PostgreSQL/SQLite.
- 9. WebSockets Broadcaster -> Streams real-time transaction events and pending approval popups to the Admin Dashboard.

Technology Stack & Toolchain Justification

- Backend: Python 3.10+, FastAPI, Starlette (Async ASGI for high concurrency and microsecond routing).
- Database & ORM: PostgreSQL & SQLite via Async SQLAlchemy (ACID audit trails with JSONB payload indexing).
- Frontend UI: Glassmorphism Single-Page Dashboard with native HTML5 WebSockets (/ws/traffic).
- Verification Hub: Built multi_mcp_hub.py validating compatibility across 15 real MCP servers (GitHub, Ruflo, Docker, etc.).

SPEAKER NOTES & FACULTY EXAMINER TALKING POINTS:

Slide 3 details our architecture. Every request passes through a 10-stage pipeline: interception, risk scoring, heuristic analysis, decision gating, rate limiting, forwarding, PII sanitization, background queue logging, and real-time dashboard broadcasting.

SLIDE 4 OF 5: Cost, Resources, Environmental Relevance & Sustainability

Parameters Covered: Cost, Resources, Environmental Relevance, Sustainability

Cost Efficiency & Zero SaaS Dependencies

- 100% Open-Source Stack: Constructed using Python, FastAPI, SQLite, and PostgreSQL with zero commercial licensing fees.
- Elimination of Third-Party API Costs: Regex automata and heuristic risk engines run locally in memory without querying costly external LLM APIs for guardrail checks, saving significant operational expenditure.

Resource Optimization & Minimal Hardware Footprint

- Lightweight Footprint: Consumes under 50 MB of memory and less than 2% CPU at idle, allowing deployment on micro-instances or edge VMs.
- Microsecond Execution: Local pattern matching executes in sub-2ms, preventing processing bottlenecks.

Environmental Relevance & Green Computing (Loop Suppression)

- Preventing AI Token Waste: Autonomous agents frequently enter infinite hallucination loops, making hundreds of redundant LLM API calls and heavy database queries.
- Compute & Energy Conservation: AgentShield's Loop Quarantine detector isolates rogue agents within 5 seconds, preventing wasted GPU cluster cycles, cloud carbon emissions, and excessive data center power draw.

Sustainability & Extensibility

- Standardized JSON-RPC 2.0 interface ensures AgentShield can govern any future MCP server release without vendor lock-in.

SPEAKER NOTES & FACULTY EXAMINER TALKING POINTS:

Slide 4 addresses sustainability and cost. By running local heuristic evaluation, AgentShield incurs zero API inspection fees. Crucially, our loop quarantine suppresses hallucination loops, saving massive amounts of GPU compute power, data center energy, and enterprise cost.

SLIDE 5 OF 5: Progress Achieved Till Mid-Sem, Remaining Roadmap & EDI Guide Discussion

Parameters Covered: Progress Done Till Now, Remaining Work for End-Sem, Additional Guide Points

Progress Accomplished Till Mid-Semester Review

- Core Security Gateway & Routing Engine operational with standard JSON-RPC 2.0 compliance.
- 3-Tier Risk Engine & Heuristics implemented (DAN prompts, SQLi, subshells, sensitive directories).
- Sliding-window Rate Limiter & Hallucination Loop Quarantine engine active.
- Real-time Human-in-the-Loop (HITL) approval manager with 60s fail-closed timeout built.
- Recursive Response PII Sanitizer operational (protects AWS keys, Aadhaar numbers, PAN, credit cards).
- Async batch audit logging pipeline and Glassmorphism Web Dashboard prototype functional.
- Rigorous Verification: 57 automated PyTest unit & integration tests passing (100% pass rate).

Remaining Roadmap for End-Semester Review

1. Production Containerization: Packaging the gateway and MCP hub into Docker Compose & Kubernetes for one-click deployment.
2. Public Demonstration Web Portal: Developing a hosted interactive webpage allowing evaluators to test live AI agent prompts against live tools in real-time.
3. Open-Source Package Publishing: Publishing AgentShield as an installable Python package (pip install agentshield) for community integration.
4. Enterprise RBAC & OAuth2 Integration: Associating agent capabilities with organizational user roles as discussed with EDI Guide Prof. Sanyukta Deshmukh.

SPEAKER NOTES & FACULTY EXAMINER TALKING POINTS:

Slide 5 covers our progress and roadmap. The core proxy, risk engine, HITL approval, PII sanitizer, and 57 tests are complete. For the end-sem, as discussed with Prof. Deshmukh, we will focus on Docker deployment, publishing the package, building a public demo web portal, and enterprise RBAC.